Course: CS 3006: Parallel and Distributed Computing

Assignment 3: Distributed Matrix Computations using MPI and OpenMP

> **Instructions & Submission Policy:**
>
> - Rename all code files according to: `ROLL-NUM-SECTION-TASK#-VERSION.EXT` (e.g., `231-0001-A-TASK1-V1.c`). Each file must contain your name, student ID, and assignment # in comments at the top.
> - Combine all work in one folder containing only code and PDF files (no binaries or executables). Rename folder as `ROLL-NUM-SECTION` (e.g., `231-0001-A`) and compress as `.zip`.
> - Submit on Google Classroom before the deadline. Late submission and plagiarism policies will be applied as described in the course outline.
> - Code with syntax errors will receive zero marks for that task. Displayed output should be well-formatted. Be prepared for viva evaluation.

Background and Motivation

Many important computations are too large for one computer. Either their data does not fit in the memory of one machine, or execution takes too long. Such computations run on a cluster: several computers (nodes) connected by a network. Cores inside one node share memory and work together using OpenMP threads. Different nodes do not share memory, so work is split among MPI processes exchanging data via messages.

This assignment applies these ideas to three computational workloads built on matrix multiplication:

- **Question 1:** Dense matrix multiplication (computing a neural network layer for multiple inputs).
- **Question 2:** Sparse matrix multiplication (computing feature updates on a social network graph).
- **Question 3:** Parallel model gradient reduction using collective operations and ring point-to-point communication.

Topics Covered

- Row-block matrix distribution & load balance strategies.
- OpenMP work-sharing (`parallel for`, static/dynamic scheduling, variable scoping).
- MPI Point-to-Point Communication (`MPI_Send`, `MPI_Recv`, non-blocking `MPI_Isend`, `MPI_Irecv`, `MPI_Waitall`, deadlock avoidance).
- MPI Collective Operations (`MPI_Bcast`, `MPI_Scatterv`, `MPI_Gatherv`, `MPI_Allgatherv`, `MPI_Reduce`, `MPI_Allreduce`, `MPI_Barrier`).
- Overlapping communication with computation using non-blocking primitives; Hybrid MPI-OpenMP programming.

Cluster Environment & Timing Rules

All work must be executed on an MPI cluster of at least two nodes. In your report, detail the number of nodes, CPU cores, memory per node, and process/thread allocations used.

Timing pattern: Measure overall runtime on the slowest process. Each process must separately track computation and communication time (time spent inside MPI calls including waiting) and gather these metrics to rank 0 using `MPI_Gather`.

```
MPI_Barrier(MPI_COMM_WORLD);
double t0 = MPI_Wtime();
/* Computation and communication only: no data generation or printing */
MPI_Barrier(MPI_COMM_WORLD);
double t_local = MPI_Wtime() - t0, t_max;
MPI_Reduce(&t_local, &t_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
if (rank == 0) printf("time: %.6f s\n", t_max);
```

Question 1: Computing a Neural-Network Layer on a Cluster

A neural network layer computes outputs as weighted sums: $y_1 = x_1 \cdot w_{11} + x_2 \cdot w_{21} + x_3 \cdot w_{31}$. For batch inputs, this is represented as dense matrix multiplication $Y = X \times W$ (or $C = A \times B$ for square $N \times N$ matrices). With $N = 32768$, double-precision storage requires 24 GiB across matrices, necessitating distributed memory across nodes.

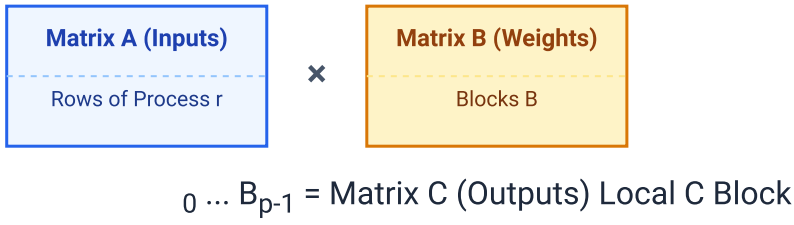


Figure 1: Row-block decomposition of matrices across p processes.

Rows of A, B, and C are split into p blocks. Process r owns block r. To calculate local rows of C, process r needs all blocks of B: $C_r = A_{r,0}B_0 + A_{r,1}B_1 + \ldots + A_{r,p-1}B_{p-1}$.

- **Version A (Broadcast):** Broadcast full matrix B to all processes via `MPI_Bcast`.
- **Version B & C (Ring Exchange):** Shift blocks of B sequentially through a logical ring network. Process r receives block B_q, computes its product term, and forwards B_q to its left neighbor while receiving from its right.

Data Generation Formula

Initialize entries locally using: $A(i, j) = ((2i + j) \bmod 7) - 3$ and $B(i, j) = ((i + 3j) \bmod 5) - 1$ for $0 \leq i, j < N$.

Tasks for Question 1

- **Task 1.1:** Compare loop orderings (i-j-k vs i-k-j) and explain why i-k-j provides superior cache performance. Parallelize i-k-j with `#pragma omp parallel for`. Report timing for $N = 1000, 2000$ across varying thread counts.
- **Task 1.2 (Version A):** Scatter A using `MPI_Scatterv`, broadcast B using `MPI_Bcast`, compute, and gather C using `MPI_Gatherv`. Benchmark timing breakdown for $N = 2000, 4000$ with $p = 1, 2, 4, 8$. Calculate memory scaling limits.
- **Task 1.3 (Version B):** Implement the blocking ring buffer exchange using `MPI_Send` and `MPI_Recv` with even/odd rank ordering to prevent deadlock.
- **Task 1.4 (Version C):** Overlap communication and computation using non-blocking `MPI_Isend`, `MPI_Irecv`, `MPI_Waitall` with dual ring buffers. Combine with OpenMP multithreading inside each process.

Question 1 Verification Tables

N	C(0,0)	C(0,1)	C(1,0)	C(N/2, N/2)	C(N-1, N-1)
1000	2	-2	7	3	9
2000	5	-10	15	-12	-5
4000	-7	5	20	9	17

Table 1: Reference matrix verification values for Question 1.

Question 2: Sparse Matrix Multiplication on a Social Network

In a social graph with n users, each user updates an feature vector $h(i)$ (size $F = 16$) by averaging feature vectors of their degree d neighbors: $h_{new}(i) = \sum_{j \in N(i)} h(j) / d$. This corresponds to sparse matrix-matrix multiplication $H_{new} = W \times H$, stored in Compressed Sparse Row (CSR) format: `row_ptr`, `col_idx`, and `val`.



Figure 2: Process memory layout showing boundary and halo region exchanges.

Graph links span at most 128 neighbors away. Each process stores its assigned user range plus 128 *halo rows* from adjacent boundary processes.

Network Generator Code Interface

```
#include <stdint.h>
static inline uint32_t hash32(uint32_t x) {
    x ^= x >> 16; x *= 0x7feb352dU;
    x ^= x >> 15; x *= 0x846ca68bU;
    x ^= x >> 16; return x;
}
static inline int degree(long i, long n) {
    uint32_t h = hash32((uint32_t)i);
    if (i < n/10 && i % 4 == 0) return 64 + (int)(h % 64); /* Popular hub user */
    return 4 + (int)(h % 9);                               /* Standard user */
}
static inline int link_offset(long i, int k) {
    uint32_t h = hash32((uint32_t)(i * 131 + k * 7 + 1));
    return (int)(h % 257) - 128;
}
```

Tasks for Question 2

- **Task 2.1:** Construct CSR tables and run $T = 10$ iterations. Compare OpenMP `schedule(static)` versus `schedule(dynamic, 64)`. Analyze load imbalance caused by high-degree hub users in the first 10% of rows.
- **Task 2.2 (Version A):** Replicate complete state matrix H across all nodes each step using `MPI_Allgatherv`. Track scaling performance up to $p = 8$.
- **Task 2.3 (Version B):** Exchange only 128 halo rows using non-blocking primitives (`MPI_Isend`, `MPI_Irecv`). Overlap communication by computing interior rows before calling `MPI_Waitall`.
- **Task 2.4:** Implement dynamic work balancing across processes by partitioning rows based on total graph edge counts rather than flat row counts.

Question 2 Verification Table

n	Total Graph Edges	H(0,0)	H(12345, 7)	H(n-1, 15)
100,000	1,020,227	0.493896795310	0.497876802886	0.497973641990
1,000,000	10,160,271	0.500882792529	0.499154198420	0.492279183027

Table 2: Verification values for Question 2 after T = 10 steps

Question 3: Parallel Gradient All-Reduce in Model Training

Data-parallel model training distributes batches of 512 examples across processes. Each process computes local weight gradients $G_l = X_l \times E_l$ across $L = 8$ layers ($m \times m$ matrix size). Gradients must be summed across all ranks (all-reduce) to keep model weights synchronized: $W_l = W_l - G_l / 65536$.

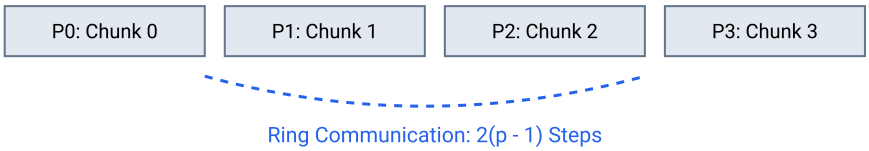


Figure 3: Ring All-Reduce phase decomposition (Reduce-Scatter followed by All-Gather).

Ring All-Reduce Protocol

Partition data into p equal chunks. Execute two phases of $p - 1$ steps:

1. **Scatter-Reduce Phase:** Send chunk $(r - s) \bmod p$ to right rank, receive from left rank, and accumulate locally.
2. **All-Gather Phase:** Send fully accumulated sum chunks around the ring to overwrite partial buffers.

Tasks for Question 3

- **Task 3.1:** Implement training loop for $S = 5$ steps. Benchmark baseline all-reduce using (a) `MPI_Reduce` + `MPI_Bcast` versus (b) standard `MPI_Allreduce`.
- **Task 3.2:** Implement custom `ring_allreduce(double *v, long n, MPI_Comm comm)` using point-to-point non-blocking transfers. Compare performance for message payloads of 1 KB, 1 MB, and 64 MB across $p = 2, 4, 8$.
- **Task 3.3:** Conduct scaling analysis for $m = 1024$. Measure speedup/efficiency limits as process counts scale while global batch size remains fixed at 512.

Question 3 Verification Tables

m	$G_0(1,2)$	$G_0(m-1,m-1)$	$G_7(m-1,m-1)$
512	33	24	27
1024	33	5	-20

Table 3(a): Gradients after step 0 all-reduce.

m	$W_0(0,0)$	$W_0(1,2)$	$W_7(m-1,m-1)$
512	0.0001220703125	-0.000152587890625	-0.00079345703125
1024	0.0001220703125	-0.000152587890625	0.0008697509765625

Table 3(b): Final weights after S = 5 steps.

Deliverables & Evaluation Rubric

Submission Requirements

- **Source Code:** Structured C/C++ source code folders (`Q1/` , `Q2/` , `Q3/`), fully commented, with build `Makefile` .
- **Technical Report:** Max 10-page technical paper documenting cluster hardware, speedup/efficiency plots, computation vs communication breakdown charts, and scaling analyses.

Evaluation Criterion	Marks
Cluster Setup & Verification	5
Q1: Loops, OpenMP, Ring Versions A/B/C	30
Q2: CSR, OpenMP, Halos, Load Balance	30
Q3: Collectives, Custom Ring, Scaling	25
Technical Report & Discussion Quality	10
Total Score	**100**